Week 10: Making the Best Decisions

Convex Optimization, Constraints, and Smart-Grid Dispatch

Instructor / Mentor

Kiki Research Training Program

Week 10

Contents

Class Mission

Week 10 Mission

Core idea

Convex optimization is a way to make the best decision when we have a goal, limits, and tradeoffs.

1. Translate a real decision into variables, an objective function, and constraints.
2. Use the "lowest point in a valley" picture to understand convexity.
3. Model a simple power-generation dispatch problem.
4. Use `scipy.optimize` to choose the lowest-cost generation mix.
5. Learn how to debug an optimizer by checking units, bounds, and feasibility.

Today's promise

By the end, students should be able to say: "The optimizer did not make magic decisions; I told it what counts as good and what rules it must obey."

120-Minute Flow

1. **0–15 min:** warm-up scenario: ordering food on a budget.
2. **15–35 min:** optimization vocabulary: variables, objectives, and constraints.
3. **35–55 min:** convexity intuition: valleys, feasible regions, and local minima.
4. **55–75 min:** smart-grid dispatch: meeting electricity demand at lowest cost.
5. **75–105 min:** Python `scipy.optimize` lab.
6. **105–115 min:** debugging and interpretation.
7. **115–120 min:** exit ticket and homework briefing.

Where This Fits in the Course

You already know

- Data can be represented with vectors.
- Matrices can describe systems.
- Graphs can represent power networks.
- Calculus helps describe change.
- Code can automate repeated calculations.

Today we add

- A mathematical language for “best.”
- A safe way to encode real-world limits.
- A bridge from math formulas to operational decisions.
- A foundation for later AI-grid control and market design.

The Big Question

How do we make a decision when there are many possible choices, but only some choices are allowed?

Goal

Minimize cost, risk, error, or emissions.

Rules

Respect budgets, capacity limits, demand, and safety.

Decision

Choose numbers that satisfy the rules and optimize the goal.

Scenario: Food Budget

Warm-Up Scenario: Ordering Food on a Budget

You are ordering food for a study group. You want the meal to be cheap, but it still has to satisfy everyone.

Food	Cost	Calories	Protein
Rice bowl	\$7	550	16 g
Salad	\$8	320	12 g
Chicken wrap	\$10	600	35 g
Fruit cup	\$4	120	2 g

Challenge

Pick quantities so the group gets enough food and protein, but spends as little money as possible.

Translate the Food Problem Into Math

Decision variables

$x_1 =$ rice bowls, $x_2 =$ salads,

$x_3 =$ wraps, $x_4 =$ fruit cups

Objective function

$$\min \ 7x_1 + 8x_2 + 10x_3 + 4x_4$$

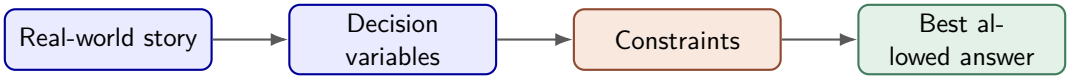
Example constraints

$$550x_1 + 320x_2 + 600x_3 + 120x_4 \geq 2400$$

$$16x_1 + 12x_2 + 35x_3 + 2x_4 \geq 100$$

$$x_1, x_2, x_3, x_4 \geq 0$$

Optimization Is Translation



Key habit

Never start by asking "Which Python function do I call?" Start by asking:

1. What can I choose?
2. What am I trying to improve?
3. What rules must not be broken?

Optimization Language

The Standard Optimization Template

Mathematical form

$$\begin{aligned} \min_{x} \quad & f(x) \\ \text{subject to} \quad & g_i(x) \le 0, \quad i = 1, 2, \ldots, m, \\ & h_j(x) = 0, \quad j = 1, 2, \ldots, p. \end{aligned}$$

x

Decision variables: the numbers we are allowed to choose.

$f(x)$

Objective function: the score we want to minimize or maximize.

g, h

Constraints: the rules that define allowed choices.

Objective Functions: What Counts as Good?

Common objectives

- Minimize money spent.
- Minimize prediction error.
- Minimize greenhouse-gas emissions.
- Minimize overload risk.
- Maximize profit, reliability, or comfort.

Smart-grid example

If generator i produces p_i MW, a simple cost model is

$$f(p) = \sum_i c_i p_i.$$

A richer model may include quadratic fuel cost:

$$f(p) = \sum_i a_i p_i^2 + b_i p_i.$$

Constraints: What Choices Are Allowed?

Equality constraints

Something must match exactly.

$$\sum_i p_i = \text{demand}$$

In the grid, total generation must match total load plus losses.

Inequality constraints

Something must stay below or above a limit.

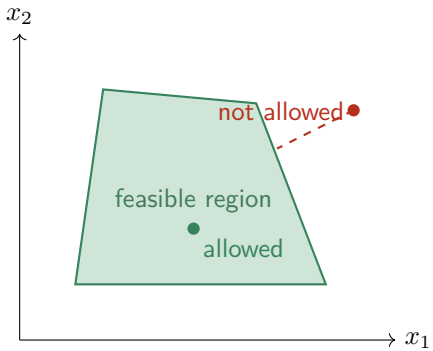
$$p_i \le P_i^{\max}, \qquad p_i \ge P_i^{\min}$$

Equipment has physical limits.

Engineering mindset

If the constraints are wrong, the optimizer may return a mathematically correct but physically useless answer.

Feasible vs. Infeasible



Feasible means legal

A solution is **feasible** if it satisfies every constraint.

Infeasible means impossible

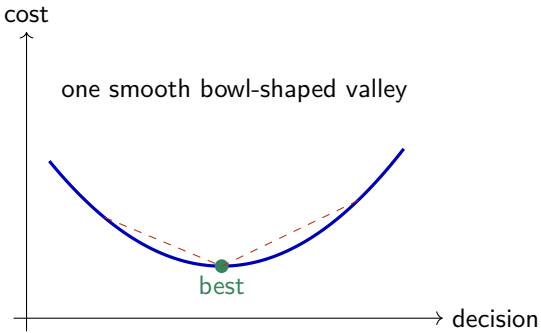
A solution is **infeasible** if it breaks at least one rule.

Power-grid example

If demand is 500 MW but total maximum generation is 430 MW, no optimizer can invent the missing capacity.

Convexity Intuition

Convexity: The Valley Picture



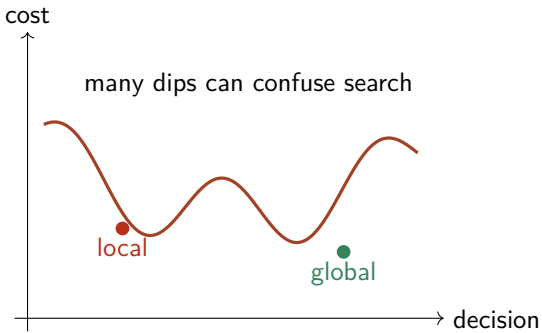
Intuition

In a convex problem, moving downhill leads toward the global best answer.

Why engineers like convexity

Convex problems are easier to solve reliably, easier to debug, and easier to trust.

Nonconvexity: Many Valleys



Local best is not always global best

In a nonconvex problem, a search method can get stuck in a nearby valley.

Today's scope

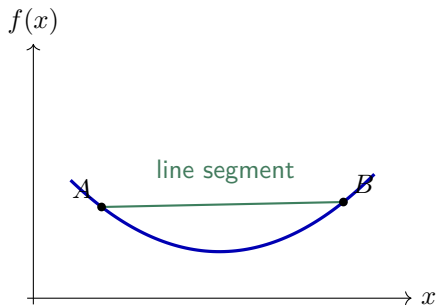
We focus on convex-style problems where the "valley" structure makes optimization more dependable.

A Visual Test for Convex Functions

Rubber-band idea

Pick any two points on the curve. If the straight line between them stays above the curve, the function behaves like a convex bowl.

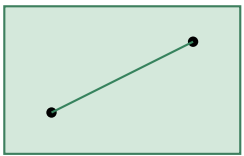
$$f(\lambda x + (1-\lambda)y) \le \lambda f(x) + (1-\lambda) f(y)$$



Convex Sets: No Holes, No Caves

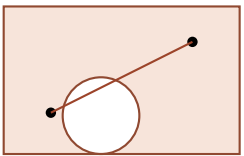
Convex feasible region

If two choices are allowed, then any mixture between them is also allowed.



Nonconvex feasible region

A mixture between two allowed choices may pass through a forbidden zone.



Convex Optimization Recipe

A convex optimization problem has two helpful properties

1. The objective function is convex, like a bowl.
2. The feasible region is convex, like a solid shape with no holes or caves.

What this buys us

If the problem is built correctly, a local minimum is also a global minimum.

Practical note

Real power-grid optimization can be more complicated than today's model. We start with a simplified convex version so the core ideas are visible.

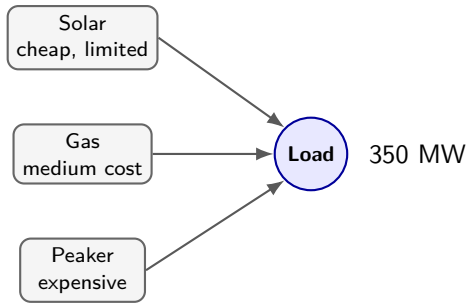
Smart-Grid Dispatch

Smart-Grid Scenario: Unit Commitment Lite

Syllabus scenario

Solve a basic power-generation decision: what is the most cost-effective mix of generators that meets demand?

- Demand must be served right now.
- Each generator has a maximum output.
- Some generators are cheap but limited.
- Some generators are expensive but flexible.



Dispatch Data for Today's Lab

Generator	Minimum MW	Maximum MW	Cost $/MWh
Solar farm	0	90	5
Wind farm	0	120	8
Gas turbine	40	220	45
Backup peaker	0	160	95

Task

Meet a demand of 350 MW at the lowest generation cost.

Decision Variables for Dispatch

Let

$$p = [p_{\text{solar}},\ p_{\text{wind}},\ p_{\text{gas}},\ p_{\text{peaker}}]$$

where each entry is the MW produced by one generator.

Objective

$$\min\ 5p_s + 8p_w + 45p_g + 95p_b$$

Power balance

$$p_s + p_w + p_g + p_b = 350$$

Bounds Are Constraints Too

Generator limits

$$0 \le p_s \le 90, \quad 0 \le p_w \le 120,$$

$$40 \le p_g \le 220, \quad 0 \le p_b \le 160.$$

Plain-English meaning

Solar and wind are cheap, so the optimizer wants to use them first. But their maximum outputs prevent them from serving the whole demand.

Manual Dispatch Reasoning

1. Use the cheapest generator first: solar up to 90 MW.
2. Use the next cheapest: wind up to 120 MW.
3. Remaining demand is $350 - 90 - 120 = 140$ MW.
4. Gas must produce at least 40 MW and can produce up to 220 MW, so gas can cover the remaining 140 MW.
5. The peaker is not needed because it is the most expensive generator.

Expected answer

$$p^* = [90,\ 120,\ 140,\ 0]$$

Why Use an Optimizer If This Example Is Easy?

Small example

Four generators, one demand number, simple linear costs.

A human can reason through it.

Real system

Thousands of generators, transmission lines, time periods, reserves, uncertainty, markets, and safety rules.

A human needs mathematical automation.

Research mindset

Start with a tiny model you can verify by hand, then scale up carefully.

From Unit Commitment to Economic Dispatch

Terminology note

Full **unit commitment** decides which generators are on/off, often using binary variables. Today's simplified version assumes the available units are already on and chooses their output levels.

Economic dispatch

Continuous variables: how many MW should each generator produce?

Full unit commitment

Binary variables: should a generator be on or off? This can become nonconvex or mixed-integer.

Adding Quadratic Fuel Costs

Linear costs are useful for intuition. Many generators are better approximated by curved cost functions:

$$C_i(p_i) = a_i p_i^2 + b_i p_i + c_i.$$

Why quadratic?

Producing one extra MW can become more expensive as a generator works harder.

Convex condition

If $a_i \geq 0$, then the cost curve bends upward like a bowl.

Python Lab

Python Lab: Core Setup

```python
import numpy as np
from scipy.optimize import minimize

names = ["solar", "wind", "gas", "peaker"]
cost = np.array([5, 8, 45, 95], dtype=float)
bounds = [(0, 90), (0, 120), (40, 220), (0, 160)]
demand = 350.0
```

Meaning

bounds stores each generator's minimum and maximum allowed MW.

Python Lab: Objective Function

```python
def total_cost(p):
    return np.sum(cost * p)
```

Mathematical translation

$$\texttt{np.sum(cost * p)} = 5p_s + 8p_w + 45p_g + 95p_b.$$

Manual coding reminder

For homework, students will fill in this logic themselves rather than copying a finished version.

Python Lab: Equality Constraint

```python
def power_balance(p):
    return np.sum(p) - demand

constraints = [
    {"type": "eq", "fun": power_balance}
]
```

How `scipy.optimize` reads this

Equality constraints should return zero when satisfied:

$$\sum_i p_i - \text{demand} = 0.$$

Python Lab: Run the Optimizer

```
x0 = np.array([50, 80, 180, 40], dtype=float)

result = minimize(
    total_cost,
    x0,
    method="SLSQP",
    bounds=bounds,
    constraints=constraints,
)

print(result.success)
print(result.message)
print(result.x)
print(total_cost(result.x))
```

Question

Does the optimizer find the same answer that we reasoned by hand?

Interpreting the Output

Checklist

1. `result.success` should be `True`.
2. `result.x` gives the generator outputs.
3. The outputs should respect every lower and upper bound.
4. The outputs should add up to demand.
5. The cost should make physical sense.

Best practice

Always verify an optimization result with independent checks, not just the success flag.

Add Your Own Verification Code

```python
p_star = result.x

print("Total generation:", np.sum(p_star))
print("Demand:", demand)
print("Balance error:", np.sum(p_star) - demand)

for name, value, limit in zip(names, p_star, bounds):
    lower, upper = limit
    print(name, value, "within", lower, "and", upper)
```

Engineering habit

Optimizers are tools. Engineers still own the responsibility for checking the answer.

Sensitivity: What If Demand Changes?

Demand	Expected expensive generator?	Why?
250 MW	No peaker	cheap plus gas is enough
350 MW	No peaker	gas covers remaining load
450 MW	Maybe peaker	cheap and gas capacity may not be enough
520 MW	Infeasible?	total max is $90 + 120 + 220 + 160 = 590$

Mini challenge

Re-run the optimizer for different demand values and explain when the backup peaker starts producing power.

Common Optimizer Bugs

Math-model bugs

- Forgetting a constraint.
- Reversing a sign.
- Using MW in one place and kW in another.
- Setting an impossible demand.

Code bugs

- Objective returns an array instead of one number.
- Constraint returns the wrong shape.
- Initial guess violates bounds badly.
- Reading output without checking `success`.

AI Collaboration: Good Prompts vs. Bad Prompts

Weak prompt

"Solve my optimization homework."

This skips thinking and creates code you may not understand.

Strong prompt

"I think my equality constraint should return zero. Here is my formula. Can you help me check whether the sign matches the math?"

This uses AI as a reasoning partner.

Mini Lab Extensions

After the basic solution works, try one extension:

1. Add a carbon-emission cost and compare the new dispatch.
2. Add a reserve requirement: total unused capacity must be at least 40 MW.
3. Use quadratic costs for gas and peaker generators.
4. Plot total cost as demand changes from 200 MW to 500 MW.
5. Simulate a cloudy day by reducing the solar maximum from 90 MW to 30 MW.

Research taste

Every extension is a new assumption. A good researcher explains assumptions clearly.

Wrap-Up and Homework

Exit Ticket

Answer these before leaving:

1. In one sentence, what is an objective function?
2. In one sentence, what is a constraint?
3. Why is convexity helpful?
4. In the dispatch problem, why do solar and wind get used before the peaker?
5. What is one check you should run after getting an optimizer result?

Homework for This Week

Manual programming training

Complete a `scipy.optimize` dispatch script with masked `TODO` sections.

Main requirements

- Implement the cost objective.
- Implement the power-balance equality constraint.
- Verify bounds and demand balance after optimization.
- Run at least three demand scenarios and interpret the results.

Feynman Technique

Write a beginner-friendly note explaining objective functions and constraints using either the food-budget example or the power-grid dispatch example.

Key Takeaways

1. Optimization means choosing decision variables to improve an objective while obeying constraints.
2. Convex problems have a helpful valley-like structure.
3. Smart-grid dispatch is an optimization problem because demand must be served under physical limits.
4. `scipy.optimize.minimize` can solve small dispatch models when objective, bounds, and constraints are encoded correctly.
5. The researcher must still check whether the optimizer's answer makes engineering sense.